

Cahier des charges

Assistant SOC intelligent basé sur les modèles de langage (LLM)

Projet de validation — Année B2

Version : 3.0

Date : Avril 2026

1. Définition du besoin

1.1 Contexte

La cybersécurité est aujourd'hui un enjeu stratégique pour toutes les organisations. Les Security Operations Centers (SOC) surveillent en permanence les systèmes d'information pour détecter et répondre aux incidents. Mais ils sont confrontés à trois défis majeurs : un **volume d'alertes en explosion**, une **difficulté d'analyse** des logs bruts et une **pénurie d'experts** estimée à 15 000 postes en France. Résultat : des incidents passent inaperçus, le temps de réaction s'allonge et les organisations restent vulnérables.

1.2 Problème

Les analystes SOC reçoivent jusqu'à plusieurs milliers d'alertes par jour. **80 % sont des faux positifs**. Les logs bruts sont illisibles, longs à analyser et exigent une forte expertise. La fatigue d'alerte conduit à des erreurs et à des oublis critiques.

1.3 Réponse apportée

Le projet propose un **assistant intelligent basé sur les LLM** qui prend en entrée des fichiers de logs et produit en sortie :

- Un **résumé en langage naturel** de chaque incident détecté.
- Un **niveau de sévérité** (Critique / Élevé / Moyen / Faible).
- Une **liste d'actions recommandées** à l'analyste.
- Un **mode conversationnel** pour poser des questions sur les logs.

1.4 Périmètre du projet

Sont **inclus** dans le périmètre : l'analyse statique de fichiers de logs, la détection d'attaques classiques, la génération de résumés et de recommandations via LLM, le stockage des incidents et l'interaction conversationnelle.

Sont **exclus** du périmètre : la connexion directe à un SIEM en production, le blocage automatique d'IP au firewall, le forensics avancé (reverse engineering, analyse mémoire), l'analyse de malwares binaires et la prise de décision autonome sans validation humaine.

2. Utilisateurs cibles

L'assistant s'adresse principalement aux **analystes SOC** de niveau 1 et 2, qui constituent la première ligne d'analyse des alertes. Il peut également être utilisé par les **responsables SOC** pour la consultation de rapports synthétiques, et dans un cadre **pédagogique** par les étudiants et formateurs en cybersécurité.

L'objectif n'est pas de remplacer ces utilisateurs, mais de leur fournir un **copilote** qui accélère leur travail et améliore la qualité de leurs décisions.

3. Contraintes

3.1 Contraintes techniques

L'assistant fonctionne sous **Linux** (Kali, Debian, Ubuntu). Il est développé en **Python 3.11+** et nécessite un accès Internet pour appeler l'API du LLM. Le volume de logs traité dans le cadre du prototype est limité à **50 Mo par fichier**.

3.2 Contraintes de performance

L'analyse d'un lot de **1 000 lignes** doit être réalisée en moins de **10 secondes**. Une réponse à une question en langage naturel doit être renvoyée en moins de **5 secondes**. L'interface ne doit jamais bloquer plus de quelques secondes.

3.3 Contraintes de sécurité

Voir le chapitre **8. Sécurité du projet** pour le détail de l'ensemble des exigences de sécurité.

4. Architecture technique

4.1 Vue d'ensemble

L'assistant est organisé en six modules indépendants, articulés en pipeline. Cette modularité permet de remplacer chaque composant sans toucher aux autres — par exemple, changer de LLM ou ajouter une nouvelle source de logs.



4.2 Choix technologiques

Composant	Technologie	Justification
Langage	Python 3.11	Standard cybersécurité et IA
LLM	API Claude (Anthropic)	Qualité d'analyse, fenêtre contextuelle large
Modèle	claude-sonnet-4-5	Bon rapport performance / coût
Base de données	SQLite	Sans serveur, fichier unique, zéro configuration
Interface	Rich (terminal)	Affichage coloré et lisible
Configuration	python-dotenv	Standard pour la gestion des secrets
Tests	pytest	Standard Python

4.3 Structure du projet

```
soc-assistant/
├── src/
│   ├── parser.py           Parsing des logs
│   ├── detector.py         Règles de détection
│   ├── enricher.py         Enrichissement IP
│   ├── llm_client.py       Appels API LLM
│   ├── prompts.py          Templates de prompts
│   ├── compressor.py       Compression des logs
│   ├── cache.py            Cache des analyses LLM
│   ├── storage.py          Accès à la base SQLite
│   └── cli.py              Interface terminal
├── data/
│   ├── logs/               Fichiers de logs analysés
│   └── threat_intel.json
├── tests/
├── main.py                 Point d'entrée
├── requirements.txt
└── .env                   Configuration (non versionné)
```

4.4 Modèle de données

Les incidents analysés sont persistés en base **SQLite** dans une table unique. La structure est volontairement simple, conforme aux besoins du prototype.

Champ	Type	Description
id	Entier	Identifiant unique auto-incrémenté
timestamp	Texte (ISO 8601)	Date et heure de détection
source_ip	Texte	Adresse IP source de l'attaque
attack_type	Texte	Type d'attaque (brute_force, sqli...)
severity	Texte	Sévérité : critical / high / medium / low
summary	Texte	Résumé généré par le LLM
raw_logs	Texte (JSON)	Échantillon des logs bruts liés
recommendation	Texte (JSON)	Liste des actions recommandées

Deux index sont créés sur les champs **severity** et **timestamp** pour optimiser les requêtes de listing et de filtrage.

5. Spécifications fonctionnelles

5.1 Formats de logs supportés

L'assistant prend en charge trois formats de logs représentatifs des sources rencontrées dans un SOC.

Format	Origine	Champs extraits
Syslog auth	Logs SSH système	timestamp, IP source, utilisateur, statut
Apache / Nginx access	Serveurs web	IP source, méthode, URL, code HTTP
Firewall iptables	Pare-feu réseau	IP source, IP destination, port, action

5.2 Types d'attaques détectées

Cinq familles d'attaques sont reconnues, par combinaison de règles statiques et d'analyse LLM.

Attaque	Méthode de détection	Seuil
Brute Force SSH	Comptage des échecs par IP	> 5 échecs / 60 secondes
Injection SQL	Reconnaissance de patterns	' OR 1=1 , UNION SELECT , --
Scan de ports	Diversité des ports cibles	> 8 ports différents / 60 secondes
Accès admin suspect	Endpoints sensibles	/admin , /wp-admin , /.env
Connexion horaire anormale	Plage horaire	Entre 02h00 et 05h00

5.3 Commandes disponibles

L'interface en ligne de commande expose les opérations suivantes : analyse d'un fichier (`analyze`), listing des incidents (`list`), affichage détaillé d'un incident (`show`), mode conversationnel (`chat`), export d'un rapport (`export`) et consultation des statistiques (`stats`).

6. Spécification du prompt LLM

6.1 Principe de fonctionnement

Le LLM reçoit pour chaque incident un **prompt système** qui définit son rôle et le format strict de sa réponse, ainsi qu'un **prompt utilisateur** contenant les données de l'incident.

Le format de réponse imposé est un **JSON strict** qui garantit l'exploitabilité automatique des sorties. Cette discipline est essentielle pour éviter les sorties libres difficiles à parser et pour maîtriser la consommation de tokens.

6.2 Structure du prompt système

Le prompt système définit quatre éléments :

- Le **rôle** : assistant SOC expert en cybersécurité.
- La **mission** : produire un résumé clair, une sévérité, un type d'attaque et trois à cinq recommandations prioritaires.
- Le **format de sortie** : JSON strict avec les champs `summary` , `severity` , `attack_type` , `recommendations` .
- Les **règles de prudence** : choisir la sévérité la plus prudente en cas de doute, ne jamais sortir du format JSON.

6.3 Construction du prompt utilisateur

Le prompt utilisateur est généré dynamiquement à partir des données de l'incident. Il contient la nature de l'attaque détectée, les caractéristiques de la source (IP, pays, réputation), les volumes observés, la période concernée et un échantillon des logs bruts associés.

Une étape de **compression** est appliquée en amont : les logs répétitifs sont regroupés sous forme `[Nx] motif` , ce qui réduit jusqu'à 70 % le nombre de tokens envoyés au LLM sans perte d'information.

6.4 Gestion des erreurs

Le client LLM intègre trois mécanismes de robustesse. En cas de **timeout** ou d'erreur réseau, le système bascule automatiquement sur un **mode dégradé** basé uniquement sur les règles statiques. En cas de **réponse mal formée**, un parser tolérant tente de récupérer le JSON. En cas de **dépassement de tokens**, les logs sont tronqués à un échantillon représentatif.

Cette architecture garantit que l'assistant reste opérationnel même si l'API du LLM est indisponible.

7. Optimisation des coûts et de la consommation

L'usage d'un LLM via API entraîne des coûts proportionnels au volume de tokens consommés. Le projet intègre trois mécanismes d'optimisation.

La **compression intelligente** des logs réduit la taille des prompts en regroupant les lignes répétitives. Sur les attaques par force brute, la réduction atteint 65 à 70 %.

Le **cache des analyses** évite de réinterroger le LLM pour des incidents similaires. Une analyse déjà produite pour une même IP, un même type d'attaque et un volume comparable est réutilisée pendant une heure.

Le **choix du modèle** s'oriente vers les versions économiques (Haiku, GPT-4o-mini) plutôt que les versions premium, sauf besoin spécifique. Un mécanisme de bascule automatique pourra être ajouté ultérieurement.

Une commande dédiée (**stats**) permet à tout moment de visualiser les tokens consommés, les économies réalisées et le coût estimé de la session.

8. Sécurité du projet

La sécurité est l'élément le plus critique de ce projet, pour deux raisons. D'abord parce qu'il s'agit d'un **outil de cybersécurité** : la confiance des utilisateurs en dépend. Ensuite parce que les **logs analysés contiennent par nature des données sensibles** : adresses IP, identifiants, requêtes vers des applications internes.

8.1 Gestion des secrets

Aucune clé d'API n'est jamais inscrite dans le code source. La clé du LLM est stockée exclusivement dans un fichier **.env** local, exclu du dépôt Git par le fichier **.gitignore**. Un fichier **.env.example** est versionné, sans valeurs réelles, pour documenter les variables attendues.

À chaque exécution, la clé est chargée en mémoire via la bibliothèque **python-dotenv**. Aucune trace n'est écrite en log ni dans la base de données. En production, une rotation périodique des clés est recommandée.

8.2 Confidentialité des données envoyées au LLM

Les logs sont par nature sensibles. Le projet intègre une option d'**anonymisation** activable au démarrage. Lorsque cette option est active :

- Les adresses IP **internes** (plages 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) sont remplacées par des hashes déterministes avant envoi au LLM.
- Les **noms d'utilisateur** présents dans les logs sont remplacés par des identifiants génériques (**user_1** , **user_2**).
- Les **logs eux-mêmes** sont tronqués à un nombre maximal de lignes par incident, pour limiter l'exposition.

Cette anonymisation préserve la pertinence de l'analyse tout en réduisant fortement la surface de risque en cas de fuite côté fournisseur LLM.

8.3 Protection des données stockées

La base SQLite est stockée localement, dans un répertoire `data/` exclu du dépôt Git. Les permissions du fichier sont restreintes au propriétaire (`chmod 600`).

Aucune donnée d'authentification (mot de passe, jeton de session) ne doit jamais être stockée. Si un log contient un tel élément, il doit être filtré en amont par le module de parsing.

8.4 Robustesse du code

Le projet est développé dans une démarche de défense en profondeur. Le parser tolère les logs corrompus sans planter — chaque ligne est traitée dans un bloc `try/except` . Les entrées utilisateur (commandes CLI, questions en mode chat) sont validées avant traitement. Les requêtes SQL utilisent exclusivement le **paramétrage** (`?`), jamais la concaténation, pour éliminer tout risque d'injection SQL.

8.5 Limitation du débit (rate limiting)

Pour éviter une surconsommation accidentelle ou malveillante de l'API LLM, un rate limiting interne est appliqué : un délai minimum est imposé entre deux appels successifs (six secondes pour OpenAI, douze secondes pour Anthropic). Cette limite peut être ajustée dans la configuration.

8.6 Journalisation et auditabilité

Toutes les opérations sensibles sont tracées : chaque analyse produit un enregistrement en base avec horodatage, IP source, type d'attaque détecté et résumé généré. Cette journalisation permet à un responsable SOC d'auditer l'usage de l'assistant et de remonter rapidement la chaîne d'événements en cas de litige.

8.7 Conformité réglementaire

Le projet respecte les principes du **RGPD** : minimisation des données envoyées au LLM, possibilité d'effacement des incidents stockés, transparence sur les traitements effectués. En production, une **analyse d'impact (PIA)** devra être conduite si l'assistant traite des logs contenant des données à caractère personnel.

L'utilisation d'un LLM hébergé hors d'Europe pose une question de **souveraineté**. Le projet est conçu pour permettre, à terme, l'usage d'un LLM local (Mistral) ou hébergé en Europe — sans modification majeure du code, grâce à l'abstraction du client LLM.

9. Analyse des risques

Risque	Probabilité	Impact	Mitigation
API LLM indisponible	Faible	Élevé	Bascule automatique en mode dégradé (règles statiques)
Coût API trop élevé	Moyenne	Moyen	Compression des logs, cache des analyses, rate limiting
Hallucinations du LLM	Moyenne	Élevé	Format JSON strict, validation humaine systématique
Faux positifs en sortie	Moyenne	Moyen	Croisement avec règles statiques, niveau de confiance affiché
Fuite de données vers le LLM	Faible	Critique	Anonymisation activable des IP internes et utilisateurs
Clé API exposée	Faible	Critique	<code>.env</code> exclu de Git, hook de pre-commit recommandé
Logs corrompus	Élevée	Faible	Parser robuste avec gestion d'exceptions
Retard de développement	Moyenne	Élevé	Découpage en sprints, MVP prioritaire

10. Planning de développement

Le projet est organisé en **sept sprints** d'une semaine chacun, selon une approche itérative inspirée de Scrum. Chaque sprint produit un incrément fonctionnel testable.

Sprint	Objectif principal
1	Cadrage, état de l'art, choix technologiques
2	Fondations : parser et jeux de logs de test
3	Détection : règles statiques et enrichissement
4	Intégration LLM, prompts, gestion d'erreurs
5	Interface terminal, mode conversationnel, export
6	Tests, optimisations (compression, cache), corrections
7	Documentation, préparation de la démonstration, soutenance

11. Critères d'acceptation

Le prototype est considéré comme validé si l'ensemble des conditions suivantes est rempli :

- L'assistant parse correctement au moins **trois formats** de logs.
- Il détecte les **cinq types d'attaques** spécifiés.
- Il produit un résumé LLM en moins de **10 secondes** pour 1 000 lignes.
- Il stocke les incidents dans la base SQLite avec succès.
- Il propose un **mode conversationnel** opérationnel.
- Il exporte un rapport au format **Markdown**.
- Il **bascule automatiquement** en mode dégradé en cas d'indisponibilité du LLM.
- Il **ne plante jamais** sur les jeux de test fournis.